

Instruções para o trabalho

Este documento descreve os requisitos do trabalho prático que cobre o estudo e a prática do paradigma lógico e da linguagem Prolog. O trabalho pode ser desenvolvido por um ou dois alunos (sozinho ou em dupla), mas nenhum envolvido está isento de entender e ser capaz de responder sobre todas as partes do trabalho.

Sistema Especialista (10 pts)

Desenvolva um sistema especialista em Prolog que seja capaz de fazer inferências e dar conselhos baseados no conhecimento adquirido sobre um assunto específico escolhido pelo aluno.

O trabalho será avaliado com base nos seguintes critérios:

- Conhecimento sobre o assunto da especialidade: normalmente isso é algo subjetivo, mas aqui a profundidade do conhecimento será tratada de forma objetiva envolvendo o número de fatos e regras projetados para alimentar a base de conhecimentos do sistema. Fatos e regras bem projetados serão a métrica.
- Representação do conhecimento: o conhecimento foi bem representado? Outra vez, fatos e regras bem projetados serão a métrica.
- Corretude da solução: o sistema cumpre o objetivo de auxiliar o usuário nas decisões relacionadas ao assunto escolhido, oferecendo boas respostas para as consultas?
- Eficiência da solução: as consultas fazem os cortes necessários para evitar explosões no tempo de execução?
- Clareza do código: fatos e regras bem projetados.
- Documentação do código: é óbvio.
- Conhecimento/exploração do Prolog. (Atente aos requisitos apresentados a seguir.)

Alguns requisitos que deverão ser cumpridos nesse trabalho:

1. Número mínimo de fatos que alimentam a base de conhecimento: 50. Trata-se de um valor mínimo para efeito de contagem, mas esteja ciente que economizar nesse quesito pode impactar negativamente na avaliação dos critérios de avaliação que foram antes apresentados, particularmente no nível de conhecimento sobre o assunto da especialidade.
2. Número mínimo de regras: 25. Mesmo comentário do item anterior.
3. Algumas regras devem realizar cortes durante a exploração. Não vou definir um número fixo, mas tenha em mente que no sistema deve transparecer que o aluno entendeu a importância de fazer cortes e como fazê-los.
4. Algumas regras devem empregar recursividade! Não vou definir um número fixo, mas tenha em mente que no sistema deve transparecer que o aluno entendeu a utilidade da recursividade e como empregá-la para implementar tarefas.

5. Algumas regras devem empregar listas! Não vou definir um número fixo, mas tenha em mente que no sistema deve transparecer que o aluno entendeu a importância das listas e como empregá-las para implementar tarefas.
6. Tente construir uma interface amigável.

Se nenhuma regra criada exigir uso de corte e/ou recursividade e/ou listas para bom funcionamento é indicativo que o aluno não explorou bem o Prolog ou o assunto da especialidade. Sem desmerecer a qualidade da base de conhecimento alimentada, considere a necessidade de criar regras que atentam aos requisitos exigidos na proposta. Tente explorar os aspectos mais relevantes do Prolog. (Dica: se sua base de conhecimento não contempla essas exigências, tente criar consultas mais avançadas para atendê-las.)

Não é obrigatório, mas se quiser desenvolver algo mais elaborado saiba que é possível fazer o python conversar com o prolog. De fato, já existem uma biblioteca python para esse fim, chamada pyswip. (Peça maiores informações, de exemplos e de usos, no chatGPT ou consulte a documentação da biblioteca.) Exemplo simples de comunicação entre python e prolog, para estimular sua curiosidade :)

```
from pyswip import Prolog

# Criando uma nova instância do Prolog
prolog = Prolog()

# Carregando uma base de conhecimento do Prolog
prolog.consult("base_conhecimento.pl")

# Realizando uma consulta no Prolog
for solucao in prolog.query("avo(X,Y)":
    print("X é avô de Y:", solucao["X"], solucao["Y"])
```

ENTREGA: são dois produtos: (1) o código fonte prolog; (2) o relatório PDF documentando seu trabalho.